

GhostNet-AI — SonarOps Agent Strategy

Autonomous Sonar Operator • Natural Language → Autonomous Sonar Workflow

Yes — **this is a very good direction for GhostNet-AI**, especially for an SIH demo.

But I would not present it simply as “**AI agent that clicks buttons on our website.**” That sounds like browser automation.

Present it as:

****GhostNet-AI Autonomous Sonar Operator****

The operator gives one natural-language instruction, and the agent executes the complete sonar-analysis workflow in the web application. The operator mainly reviews the final findings and only gets involved when the agent encounters uncertainty, missing data, or a decision requiring human approval.

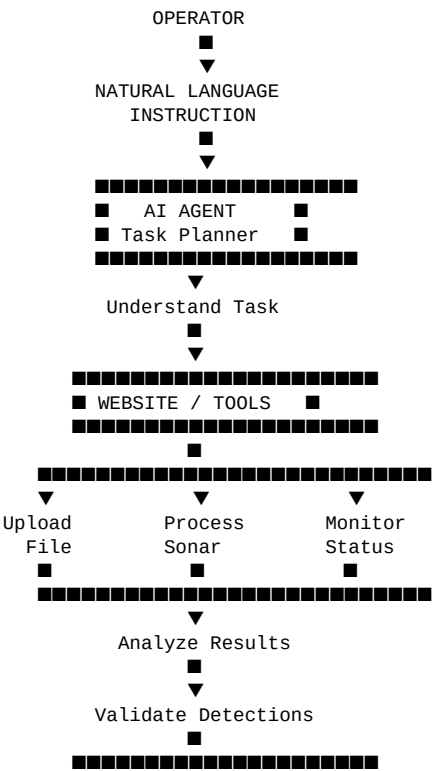
That fits very well with the human-in-the-loop concept already present in your PPT.

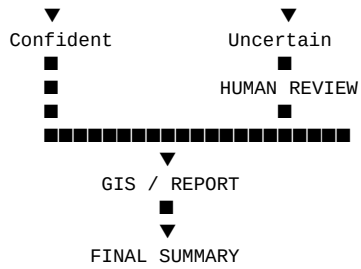
How your feature should work

The operator could type:

"Process survey_014.xtf. Detect ghost nets and other man-made objects, ignore detections below 70% confidence, map the results, and prepare the investigation report."

Then:





The really interesting part

Don't make the agent stop after uploading the file.

Make it **observe the workflow and decide what to do next**.

For example:

Step 1

Agent checks the file.

```
`survey_014.xtf detected`
```

Step 2

It checks metadata.

```
`GPS metadata: available`
```

Step 3

It checks sonar quality.

```
`Port channel: good`
```

```
`Starboard channel: moderate`
```

Step 4

It decides the processing path.

```
`Running standard preprocessing`
```

Step 5

It starts inference.

```
`12,481 tiles processed`
```

Step 6

It detects 47 candidates.

Step 7

It removes duplicates using track-level consolidation.

```
`47 detections → 19 unique targets`
```

Step 8

It sends uncertain targets to review.

```
`4 targets require human verification`
```

Step 9

It generates the final result.

```
**19 unique targets identified**
```

```
**11 high-confidence**
```

```
**4 review required**
```

```
**4 low-confidence/unknown**
```

That is much more impressive than just:

```
"AI uploaded the file."
```

Give the agent actual tools

Your agent should have a controlled toolset.

Agent

```
■  
■■■ upload_file()  
■■■ inspect_metadata()  
■■■ assess_sonar_quality()  
■■■ preprocess_sonar()  
■■■ run_detection()  
■■■ run_verification()  
■■■ track_targets()  
■■■ calculate_geolocation()  
■■■ calculate_uncertainty()  
■■■ filter_targets()  
■■■ open_target_dossier()  
■■■ request_human_review()  
■■■ generate_report()  
■■■ export_results()
```

The agent chooses these tools according to the user's instruction.

This is what makes it an **agentic workflow** rather than a chatbot.

— — —

Add one killer feature: Agent watches the result

Suppose the AI detects:

Target #17
Ghost Net
Confidence: 76%

The agent shouldn't blindly accept it.

It can inspect:

- Object evidence
- Shadow evidence
- Track consistency
- Sonar quality
- Nearby seabed context

Then decide:

76% model confidence
+ strong shadow
+ repeated in 8 frames
+ good sonar quality
→ SEND TO HUMAN REVIEW

Or:

91% model confidence
but
poor sonar quality
and
weak shadow
→ UNCERTAIN

This connects your **agent** with your **evidence-fusion / uncertainty** idea.

That's where the concept becomes much stronger.

— — —

Give the operator an "Agent Inbox"

This is something I strongly recommend.

Instead of making the operator watch everything, the agent produces an inbox:

[illegible]

Operator sees only the things that require attention.

Normal case

Agent works → operator reviews final summary

Exception case

Agent stops → operator makes decision → agent continues

That is a very clean human-AI workflow.

Another excellent feature: "Ask the Agent"

After processing, operator can ask:

***"Show me all high-confidence ghost-net detections within 10 meters uncertainty."**

The agent performs the filtering.

Or:

***"Why was Target 24 not automatically confirmed?"**

Agent answers:

"Target 24 was classified as uncertain because the object confidence was 81%, but acoustic-shadow evidence was weak and the sonar segment had moderate signal quality."

Or:

***"Generate a recovery-priority report for confirmed ghost-net targets."**

The agent generates it.

Now your interface becomes a **natural-language control layer over the entire platform**.

Even better: Agent can recover from errors

This can make your demo memorable.

Suppose:

UPLOAD
↓
ERROR: GPS metadata unavailable

A normal application says:

Error.

Your agent says:

***"GPS metadata is missing. I can still run object detection, but exact geolocation cannot be established. Continue with image-space detection?"**

Buttons:

CONTINUE

CANCEL

That is what I would call:

****Graceful Agentic Degradation****

It fits the graceful-degradation idea already in your presentation.

Very important architecture decision

There are actually **two ways** to build your idea.

Option A — Agent controls the website visually

```
AI Agent
  ↓
Browser
  ↓
See webpage
  ↓
Click Upload
  ↓
Select File
  ↓
Click Process
  ↓
Read Results
```

This is the exact thing you're imagining.

For the **hackathon demo**, this can look very impressive.

Option B — Agent controls your platform through tools/API

```
AI Agent
  ↓
Tool Layer
  ■■■ Upload API
  ■■■ Processing API
  ■■■ Detection API
  ■■■ GIS API
  ■■■ Reporting API
```

This is much more reliable.

My recommendation

Build **both conceptually**, but use:

****API/tool execution for the real system****

and

****browser interaction for the visible demo****

So the judge can literally watch:

Agent: "I'll process survey_014.xtf."

Browser opens.

Agent uploads file.

Agent starts processing.

Agent waits.

Agent inspects results.

Agent filters targets.

Agent opens GIS.

Agent generates report.

Agent returns:

"19 targets found; 4 require review."

That creates the **wow factor** without making your entire backend dependent on fragile visual clicking.

Make the agent "agent-ready"

This is an important engineering trick.

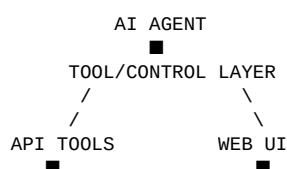
Give every important UI element a stable semantic identifier:

```
data-agent-action=&quot;upload-sonar&quot;;
data-agent-action=&quot;start-processing&quot;;
data-agent-action=&quot;open-map&quot;;
data-agent-action=&quot;generate-report&quot;;
data-agent-action=&quot;review-target&quot;;
```

Then the browser agent has a reliable interface.

Even better, expose corresponding backend tools.

So:



During execution:

- ✓ Validate
- ✓ Preprocess
- ✓ Detect
- ✓ Verify
- Geolocation
- Report

This will look excellent in the demo.

Give the agent permission levels

This is particularly important.

AUTO

Safe actions:

- upload
- preprocess
- run inference
- filter
- generate draft

APPROVAL REQUIRED

Higher-impact actions:

- permanently delete data
- overwrite a survey
- publish results
- finalize an official report
- mark a target as operationally confirmed

So the agent isn't given unrestricted control.

I would name the feature

Instead of just:

AI Agent

Use:

****GhostNet-AI Mission Agent****

or

****SonarOps Agent****

or

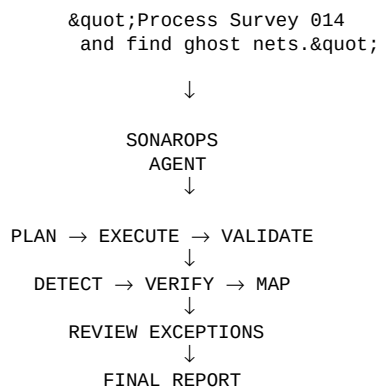
****Autonomous Sonar Analyst****

My preference for your presentation:

****SonarOps Agent****

Natural Language → Autonomous Sonar Workflow

Then a beautiful visual:



This can become your strongest differentiation

Your project then has **two AI layers**:

